Universal Biometric Device Integration — HRMS Attendance System

> ***System:*** *Multi-Tenant Spring Boot HRMS (* `com.example.multi_tanent` *)* ***Device Reference:*** *Team Office Z900 (Face + Fingerprint + Wi-Fi/TCP-IP)* ***Protocol Support:*** *TCP/IP Push, HTTP REST Push, USB Pull (ZKTeco SDK compatible)* ***Author:*** *Architecture Document | May 2026*

Overview

This document describes a **phase-wise implementation plan** to connect **any biometric device** (fingerprint, face, card, or combined) to your existing multi-tenant HRMS for automated attendance recording.

Your system already has:

- ✅ `BiometricTenantFilter.java` — tenant resolver for `/api/biometric-punch/**`
- ✅ `/api/biometric-punch/**` — whitelisted in `SecurityConfig.java` (no JWT needed)
- ✅ `Employee` entity with `employeeCode` (unique per tenant)
- ✅ `TenantContext` — multi-tenant data isolation
- ✅ Attendance module with `/api/attendance-records/**`

Architecture Diagram

```
[ Biometric Device ]
        |
  (Push via HTTP POST / TCP Socket)
        |
        ▼
[ Spring Boot Backend ]
  /api/biometric-punch/{tenantId}/punch
        |
  BiometricTenantFilter → sets TenantContext
        |
  BiometricPunchController
        |
  BiometricAttendanceService
        |
  ┌─────────────┬──────────────┐
  ▼             ▼              ▼
Employee   AttendanceRecord  ShiftPolicy
(lookup)   (save CLOCK_IN    (calculate
            / CLOCK_OUT)      late/early)
```

Phase 1 — Database Schema Design

1.1 New Table: `biometric_devices`

This table registers every physical device linked to a tenant and branch.

```
CREATE TABLE biometric_devices (
    id               BIGINT AUTO_INCREMENT PRIMARY KEY,
    tenant_id        VARCHAR(100)  NOT NULL,   -- links to TenantContext
    device_serial    VARCHAR(100)  NOT NULL UNIQUE,
```

```
    device_name      VARCHAR(150),
    device_type      VARCHAR(50),   -- FINGERPRINT | FACE | CARD | COMBINED
    protocol         VARCHAR(50),   -- HTTP_PUSH | TCP_PUSH | USB_PULL
    ip_address       VARCHAR(50),
    port             INT,
    location_id      BIGINT,        -- FK → locations table
    is_active        BOOLEAN DEFAULT TRUE,
    api_key          VARCHAR(255),  -- secret for device authentication
    created_at       DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

1.2 New Table: `biometric_raw_logs`

Stores every raw punch event exactly as received from the device — before processing.

```
CREATE TABLE biometric_raw_logs (
    id               BIGINT AUTO_INCREMENT PRIMARY KEY,
    tenant_id        VARCHAR(100)  NOT NULL,
    device_serial    VARCHAR(100),
    employee_code    VARCHAR(100),  -- device-side ID mapped to employeeCode
    biometric_uid    VARCHAR(100),  -- raw UID from device
    punch_time       DATETIME       NOT NULL,
    punch_type       VARCHAR(20),   -- IN | OUT | UNKNOWN
    verification_type VARCHAR(30),  -- FINGERPRINT | FACE | CARD | PASSWORD
    raw_payload      TEXT,          -- full JSON from device
    processed        BOOLEAN DEFAULT FALSE,
    processed_at     DATETIME,
    error_message    VARCHAR(500),
    created_at       DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

1.3 Modify Table: `attendance_records`

Add biometric tracking columns to your existing attendance table.

```
ALTER TABLE attendance_records
    ADD COLUMN biometric_device_serial VARCHAR(100),
    ADD COLUMN biometric_raw_log_id    BIGINT,
    ADD COLUMN punch_source            VARCHAR(30) DEFAULT 'MANUAL',
    -- punch_source: BIOMETRIC | MANUAL | MOBILE | SYSTEM
    ADD COLUMN clock_in_method         VARCHAR(30),
    ADD COLUMN clock_out_method        VARCHAR(30);
    -- method: FINGERPRINT | FACE | CARD | PASSWORD | MANUAL
```

1.4 New Table: `employee_biometric_mapping`

Maps each employee's biometric UID on a device to their `employeeCode`.

```
CREATE TABLE employee_biometric_mapping (
    id               BIGINT AUTO_INCREMENT PRIMARY KEY,
    tenant_id        VARCHAR(100) NOT NULL,
    employee_code    VARCHAR(100) NOT NULL,
    device_serial    VARCHAR(100) NOT NULL,
    biometric_uid    VARCHAR(100) NOT NULL,  -- user ID stored on device
    enrolled_at      DATETIME,
```

```
    is_active        BOOLEAN DEFAULT TRUE,
    UNIQUE KEY uq_device_uid (device_serial, biometric_uid)
);
```

Phase 2 — Backend: Entity & Repository Layer

2.1 Entity: `BiometricDevice.java`

Package: `com.example.multi_tanent.spersusers.enitity`

```
@Entity
@Table(name = "biometric_devices")
@Getter @Setter
public class BiometricDevice {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "tenant_id", nullable = false)
    private String tenantId;

    @Column(name = "device_serial", nullable = false, unique = true)
    private String deviceSerial;

    private String deviceName;

    @Enumerated(EnumType.STRING)
    private DeviceType deviceType;   // FINGERPRINT, FACE, CARD, COMBINED

    @Enumerated(EnumType.STRING)
    private DeviceProtocol protocol; // HTTP_PUSH, TCP_PUSH, USB_PULL

    private String ipAddress;
    private Integer port;

    @ManyToOne
    @JoinColumn(name = "location_id")
    private Location location;

    private Boolean isActive = true;

    @Column(name = "api_key")
    private String apiKey;

    @Column(name = "created_at")
    private LocalDateTime createdAt;
}
```

2.2 Entity: `BiometricRawLog.java`

```
@Entity
@Table(name = "biometric_raw_logs")
@Getter @Setter
public class BiometricRawLog {
```

```
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "tenant_id")
    private String tenantId;

    @Column(name = "device_serial")
    private String deviceSerial;

    @Column(name = "employee_code")
    private String employeeCode;

    @Column(name = "biometric_uid")
    private String biometricUid;

    @Column(name = "punch_time")
    private LocalDateTime punchTime;

    @Enumerated(EnumType.STRING)
    @Column(name = "punch_type")
    private PunchType punchType;     // IN, OUT, UNKNOWN

    @Column(name = "verification_type")
    private String verificationType; // FINGERPRINT, FACE, CARD

    @Column(name = "raw_payload", columnDefinition = "TEXT")
    private String rawPayload;

    private Boolean processed = false;

    @Column(name = "processed_at")
    private LocalDateTime processedAt;

    @Column(name = "error_message")
    private String errorMessage;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    @PrePersist
    public void prePersist() {
        createdAt = LocalDateTime.now();
        tenantId = TenantContext.getTenantId();
    }
}
```

2.3 Enums

Package: `com.example.multi_tanent.spersusers.enums`

```
// DeviceType.java
public enum DeviceType {
    FINGERPRINT, FACE, CARD, COMBINED
}
```

```
// DeviceProtocol.java
public enum DeviceProtocol {
    HTTP_PUSH,   // Device sends HTTP POST to your server
    TCP_PUSH,    // Device sends raw TCP packets (ZKTeco native)
    USB_PULL     // Your server polls device via USB/SDK
}

// PunchType.java
public enum PunchType {
    IN, OUT, UNKNOWN
}
```

2.4 Repositories

```
// BiometricRawLogRepository.java
@Repository
public interface BiometricRawLogRepository extends JpaRepository<BiometricRawLog, Long> {

    List<BiometricRawLog> findByProcessedFalseOrderByCreatedAtAsc();

    List<BiometricRawLog> findByTenantIdAndEmployeeCodeAndPunchTimeBetween(
        String tenantId, String employeeCode,
        LocalDateTime from, LocalDateTime to
    );
}

// BiometricDeviceRepository.java
@Repository
public interface BiometricDeviceRepository extends JpaRepository<BiometricDevice, Long> {

    Optional<BiometricDevice> findByDeviceSerial(String serial);

    List<BiometricDevice> findByTenantIdAndIsActiveTrue(String tenantId);
}
```

Phase 3 — Backend: API Controller

3.1 Universal Punch Endpoint

Package: `com.example.multi_tanent.spersusers.controller` **File:** `BiometricPunchController.java`

This is the single endpoint that receives punches from **any brand** of device.

```
@RestController
@RequestMapping("/api/biometric-punch")
public class BiometricPunchController {

    private final BiometricAttendanceService biometricService;

    public BiometricPunchController(BiometricAttendanceService biometricService) {
        this.biometricService = biometricService;
    }

    /**
```

```
     * Universal punch endpoint.
     * URL: POST /api/biometric-punch/{tenantId}/punch
     *
     * BiometricTenantFilter already sets TenantContext from {tenantId}.
     * No JWT needed — device uses X-Device-Api-Key header.
     *
     * Payload example:
     * {
     *   "deviceSerial": "ZK-ABC123",
     *   "employeeCode": "EMP-001",   // or biometricUid if not mapped
     *   "biometricUid": "101",
     *   "punchTime": "2026-05-02T09:05:00",
     *   "punchType": "IN",            // IN | OUT | UNKNOWN
     *   "verificationType": "FINGERPRINT"
     * }
     */
    @PostMapping("/{tenantId}/punch")
    public ResponseEntity<?> receivePunch(
            @PathVariable String tenantId,
            @RequestHeader(value = "X-Device-Api-Key", required = false) String apiKey,
            @RequestBody BiometricPunchRequest request) {

        // 1. Validate API key (device authentication)
        biometricService.validateDeviceApiKey(request.getDeviceSerial(), apiKey);

        // 2. Save raw log
        BiometricRawLog log = biometricService.saveRawLog(request);

        // 3. Process and create attendance record
        biometricService.processAndRecord(log);

        return ResponseEntity.ok(Map.of(
            "status", "SUCCESS",
            "logId", log.getId(),
            "message", "Punch recorded"
        ));
    }

    /**
     * Health check — device can ping this to verify connectivity.
     * URL: GET /api/biometric-punch/{tenantId}/ping
     */
    @GetMapping("/{tenantId}/ping")
    public ResponseEntity<?> ping(@PathVariable String tenantId) {
        return ResponseEntity.ok(Map.of("status", "OK", "tenant", tenantId));
    }

    /**
     * Device Management — list all registered devices for a tenant.
     * URL: GET /api/biometric-punch/{tenantId}/devices
     * Requires JWT (used by HRMS admin panel).
     */
    @GetMapping("/{tenantId}/devices")
    public ResponseEntity<?> listDevices(@PathVariable String tenantId) {
        return ResponseEntity.ok(biometricService.getDevicesForTenant(tenantId));
    }
```

```
    /**
     * Register a new biometric device.
     * URL: POST /api/biometric-punch/{tenantId}/devices
     */
    @PostMapping("/{tenantId}/devices")
    public ResponseEntity<?> registerDevice(
            @PathVariable String tenantId,
            @RequestBody BiometricDeviceRequest request) {
        return ResponseEntity.ok(biometricService.registerDevice(tenantId, request));
    }
}
```

3.2 Request DTOs

```
// BiometricPunchRequest.java
@Getter @Setter
public class BiometricPunchRequest {
    private String deviceSerial;
    private String employeeCode;   // preferred — direct match to Employee.employeeCode
    private String biometricUid;   // fallback — lookup via employee_biometric_mapping
    private LocalDateTime punchTime;
    private String punchType;      // "IN" | "OUT" | "UNKNOWN"
    private String verificationType; // "FINGERPRINT" | "FACE" | "CARD" | "PASSWORD"
}

// BiometricDeviceRequest.java
@Getter @Setter
public class BiometricDeviceRequest {
    private String deviceSerial;
    private String deviceName;
    private String deviceType;
    private String protocol;
    private String ipAddress;
    private Integer port;
    private Long locationId;
}
```

Phase 4 — Backend: Core Service Logic

File: `BiometricAttendanceService.java` **Package:** `com.example.multi_tanent.spersusers.service`

```
@Service
@Transactional
public class BiometricAttendanceService {

    private final BiometricRawLogRepository rawLogRepo;
    private final BiometricDeviceRepository deviceRepo;
    private final EmployeeRepository employeeRepo;
    private final AttendanceRecordRepository attendanceRepo;

    // ── Step 1: Validate Device API Key ──────────────────────────
    public void validateDeviceApiKey(String deviceSerial, String apiKey) {
        if (deviceSerial == null) return; // allow if no serial (lenient mode)
```

```
        deviceRepo.findByDeviceSerial(deviceSerial).ifPresent(device -> {
            if (device.getApiKey() != null && !device.getApiKey().equals(apiKey)) {
                throw new ResponseStatusException(HttpStatus.UNAUTHORIZED,
                    "Invalid device API key for serial: " + deviceSerial);
            }
        });
    }

    // ── Step 2: Save Raw Log ──────────────────────────────────────
    public BiometricRawLog saveRawLog(BiometricPunchRequest request) {
        BiometricRawLog log = new BiometricRawLog();
        log.setDeviceSerial(request.getDeviceSerial());
        log.setEmployeeCode(request.getEmployeeCode());
        log.setBiometricUid(request.getBiometricUid());
        log.setPunchTime(request.getPunchTime() != null
            ? request.getPunchTime() : LocalDateTime.now());
        log.setPunchType(parsePunchType(request.getPunchType()));
        log.setVerificationType(request.getVerificationType());
        log.setRawPayload(request.toString());
        log.setProcessed(false);
        return rawLogRepo.save(log);
    }

    // ── Step 3: Resolve Employee & Record Attendance ──────────────
    public void processAndRecord(BiometricRawLog log) {
        try {
            // 3a. Resolve employee code
            String empCode = resolveEmployeeCode(log);
            if (empCode == null) {
                markFailed(log, "Employee not found for biometricUid: " +
log.getBiometricUid());
                return;
            }

            Employee employee = employeeRepo.findByEmployeeCode(empCode).orElse(null);
            if (employee == null) {
                markFailed(log, "No employee with code: " + empCode);
                return;
            }

            // 3b. Find today's existing attendance record
            LocalDate today = log.getPunchTime().toLocalDate();
            AttendanceRecord existing = attendanceRepo
                .findByEmployeeCodeAndDate(empCode, today)
                .orElse(null);

            if (existing == null) {
                // First punch of the day → CLOCK IN
                AttendanceRecord record = new AttendanceRecord();
                record.setEmployeeCode(empCode);
                record.setDate(today);
                record.setClockIn(log.getPunchTime());
                record.setPunchSource("BIOMETRIC");
                record.setClockInMethod(log.getVerificationType());
                record.setBiometricDeviceSerial(log.getDeviceSerial());
                record.setBiometricRawLogId(log.getId());
```

```
                attendanceRepo.save(record);

            } else if (existing.getClockOut() == null
                    && log.getPunchTime().isAfter(existing.getClockIn())) {
                // Second punch → CLOCK OUT
                existing.setClockOut(log.getPunchTime());
                existing.setClockOutMethod(log.getVerificationType());
                attendanceRepo.save(existing);
            }
            // else: duplicate punch — ignore

            markProcessed(log);

        } catch (Exception e) {
            markFailed(log, e.getMessage());
        }
    }

    // ─── Helpers ──────────────────────────────────────────────────
    private String resolveEmployeeCode(BiometricRawLog log) {
        // Priority 1: direct employeeCode from device
        if (log.getEmployeeCode() != null && !log.getEmployeeCode().isBlank()) {
            return log.getEmployeeCode();
        }
        // Priority 2: lookup via biometric UID mapping table
        if (log.getBiometricUid() != null && log.getDeviceSerial() != null) {
            return employeeBiometricMappingRepo
                .findByDeviceSerialAndBiometricUid(log.getDeviceSerial(), log.getBiometricUid())
                .map(EmployeeBiometricMapping::getEmployeeCode)
                .orElse(null);
        }
        return null;
    }

    private PunchType parsePunchType(String type) {
        if (type == null) return PunchType.UNKNOWN;
        return switch (type.toUpperCase()) {
            case "IN", "0", "CHECK-IN" -> PunchType.IN;
            case "OUT", "1", "CHECK-OUT" -> PunchType.OUT;
            default -> PunchType.UNKNOWN;
        };
    }

    private void markProcessed(BiometricRawLog log) {
        log.setProcessed(true);
        log.setProcessedAt(LocalDateTime.now());
        rawLogRepo.save(log);
    }

    private void markFailed(BiometricRawLog log, String reason) {
        log.setProcessed(false);
        log.setErrorMessage(reason);
        rawLogRepo.save(log);
    }
}
```

Phase 5 — Device Configuration

5.1 HTTP Push Mode (Wi-Fi Devices like Z900)

Most modern devices support **HTTP Push** — the device acts as an HTTP client and sends punches directly to your server URL.

Steps on the device (via web admin panel or keypad menu):

Setting	Value
Server URL	`https://yourdomain.com/api/biometric-punch/{tenantId}/punch`
Request Method	`POST`
Content-Type	`application/json`
Custom Header	`X-Device-Api-Key: YOUR_SECRET_KEY`
Push Interval	Every punch (real-time) or every 30s

> ***For Team Office Z900:*** *Go to device web panel → `Communication` → `HTTP Push Settings` → enter the URL above.*

Sample payload your server will receive:

```
{
  "deviceSerial": "Z900-SN-00123",
  "biometricUid": "101",
  "punchTime": "2026-05-02T09:05:33",
  "punchType": "IN",
  "verificationType": "FACE"
}
```

5.2 TCP/IP Native Protocol (ZKTeco Devices)

ZKTeco and similar devices use a **proprietary binary TCP protocol** on port `4370`.

Your backend needs a **TCP Socket Listener** running as a background service:

```
// BiometricTcpListenerService.java
@Service
public class BiometricTcpListenerService implements CommandLineRunner {

    @Override
    public void run(String... args) throws Exception {
        // Start TCP server on port 4370 in background thread
        new Thread(() -> {
            try (ServerSocket serverSocket = new ServerSocket(4370)) {
                while (true) {
                    Socket client = serverSocket.accept();
                    new Thread(new DeviceConnectionHandler(client)).start();
                }
            } catch (IOException e) {
                log.error("TCP Listener error", e);
            }
        }).start();
```

```
    }
}
```

> ***Note:*** *For ZKTeco devices, use the open-source* ***zklib*** *Java library or the official* ***ZKAccess SDK*** *to parse binary packets.*

5.3 USB Pull Mode (Offline Devices)

For offline devices without network:

1. Device stores punches in internal memory.
2. Admin connects USB → runs a scheduled sync job.
3. Your backend exposes:

   ```
   POST /api/biometric-punch/{tenantId}/bulk-sync
   Body: [ { punchRecord1 }, { punchRecord2 }, ... ]
   ```

4. Service iterates and processes each record using the same `processAndRecord()` logic.

Phase 6 — Multi-Tenant Routing

Your `BiometricTenantFilter` already handles this, but here's how it maps:

```
Device pushes to:
  POST /api/biometric-punch/tenant-abc/punch
                               ↑
                          This part
                       is your tenantId

BiometricTenantFilter intercepts → calls TenantContext.setTenantId("tenant-abc")
→ All JPA queries now route to the correct tenant database schema
→ Employee lookup, AttendanceRecord save — all isolated per tenant
```

Each company (tenant) configures their own:

- Unique tenant URL segment
- Their own device serial numbers
- Their own employee codes

Phase 7 — Frontend: Device Management Panel

Add a **Biometric Devices** tab in your HRMS Settings or Attendance module.

7.1 Features Needed

Feature	Description
Register Device	Form: serial, name, type, protocol, IP, location
List Devices	Table with status, IP, location, last sync time
Test Connection	Ping button → calls `/api/biometric-punch/{tenantId}/ping`
Raw Log Viewer	Table of recent punches with processed status
Employee Mapping	Map employee → biometric UID per device

Generate API Key	One-click generate & copy secret key for device config

7.2 Sample API Calls (Frontend → Backend)

```
// Register a device
POST /api/biometric-punch/{tenantId}/devices
Authorization: Bearer {jwt}
Body: { deviceSerial, deviceName, deviceType, protocol, ipAddress, locationId }

// Get all devices
GET /api/biometric-punch/{tenantId}/devices

// View raw punch logs
GET /api/attendance-records/biometric-logs?date=2026-05-02&processed=false

// Map employee to biometric UID
POST /api/biometric-punch/{tenantId}/employee-mapping
Body: { employeeCode, deviceSerial, biometricUid }
```

Phase 8 — Attendance Auto-Processing Rules

These rules run inside `BiometricAttendanceService.processAndRecord()`:

Rule	Logic
CLOCK_IN detection	First punch of the day for employee → mark as IN
CLOCK_OUT detection	Second punch > first punch time → mark as OUT
Duplicate punch	Same punch within ±5 min window → ignore
Unknown punch type	If device sends `punchType: UNKNOWN` → auto-detect from existing record
Late arrival	Compare `clockIn` with `ShiftPolicy.startTime` → flag as LATE
Early departure	Compare `clockOut` with `ShiftPolicy.endTime` → flag as EARLY_LEAVE
Absent marking	Scheduled job at end-of-day → mark employees with no punch as ABSENT

Phase 9 — Security Considerations

Concern	Solution
Device authentication	Each device gets a unique `X-Device-Api-Key` stored in `biometric_devices.api_key`
No JWT required	`/api/biometric-punch/**` is already `permitAll()` — devices cannot hold JWTs
API key rotation	Admin can regenerate key from HRMS panel; old key invalidated
IP whitelisting	Optionally store `ipAddress` in `BiometricDevice` and validate `request.getRemoteAddr()`
Replay attacks	Check `biometric_raw_logs` for duplicate (deviceSerial + biometricUid + punchTime)
Data isolation	`TenantContext` ensures cross-tenant data leaks are impossible at JPA level
HTTPS only	Device must push to HTTPS endpoint — configure your server SSL

Phase 10 — Testing & Verification

10.1 Manual API Test (Simulate a Device Push)

```
# Simulate a biometric punch using curl
curl -X POST https://yourdomain.com/api/biometric-punch/tenant-abc/punch \
  -H "Content-Type: application/json" \
  -H "X-Device-Api-Key: YOUR_SECRET" \
  -d '{
    "deviceSerial": "Z900-TEST-001",
    "biometricUid": "101",
    "employeeCode": "EMP-001",
    "punchTime": "2026-05-02T09:05:00",
    "punchType": "IN",
    "verificationType": "FINGERPRINT"
  }'

# Expected response:
# { "status": "SUCCESS", "logId": 1, "message": "Punch recorded" }
```

10.2 Verification Checklist

- ☐ `biometric_raw_logs` table has a new row
- ☐ `attendance_records` table has a new row for the employee
- ☐ `processed = true` in raw log after processing
- ☐ Second punch updates `clock_out` on same attendance record
- ☐ Wrong tenant ID returns 404 or empty response
- ☐ Wrong API key returns 401 Unauthorized
- ☐ Duplicate punch within 5 minutes is ignored

10.3 Device Connectivity Test

1. Configure device web panel with your server URL
2. Press finger / show face on device
3. Check raw logs in HRMS admin panel
4. Verify attendance record created for that employee

Implementation Order (Recommended)

Order	Task	Effort
1	Create Liquibase migrations for 3 new tables	1 day
2	Create entities, enums, repositories	1 day
3	Implement `BiometricAttendanceService`	2 days
4	Implement `BiometricPunchController`	1 day
5	Configure first device (HTTP Push mode)	0.5 day
6	Test with curl → verify raw logs + attendance	0.5 day
7	Build frontend Device Management panel	2 days

8	Add auto-processing rules (late/absent)	1 day
9	TCP listener (if ZKTeco native protocol needed)	2 days
Total		**~11 days**

Summary

Component	File/Location
Tenant filter (existing)	`config/BiometricTenantFilter.java`
Security whitelist (existing)	`config/SecurityConfig.java` line 61
New: punch controller	`controller/BiometricPunchController.java`
New: core service	`service/BiometricAttendanceService.java`
New: entities	`enitity/BiometricDevice.java`, `BiometricRawLog.java`
New: enums	`enums/DeviceType.java`, `DeviceProtocol.java`, `PunchType.java`
New: DB migrations	`resources/db/changelog/` (Liquibase)
New: frontend panel	`frontend/src/hrms/biometric/BiometricDevices.jsx`